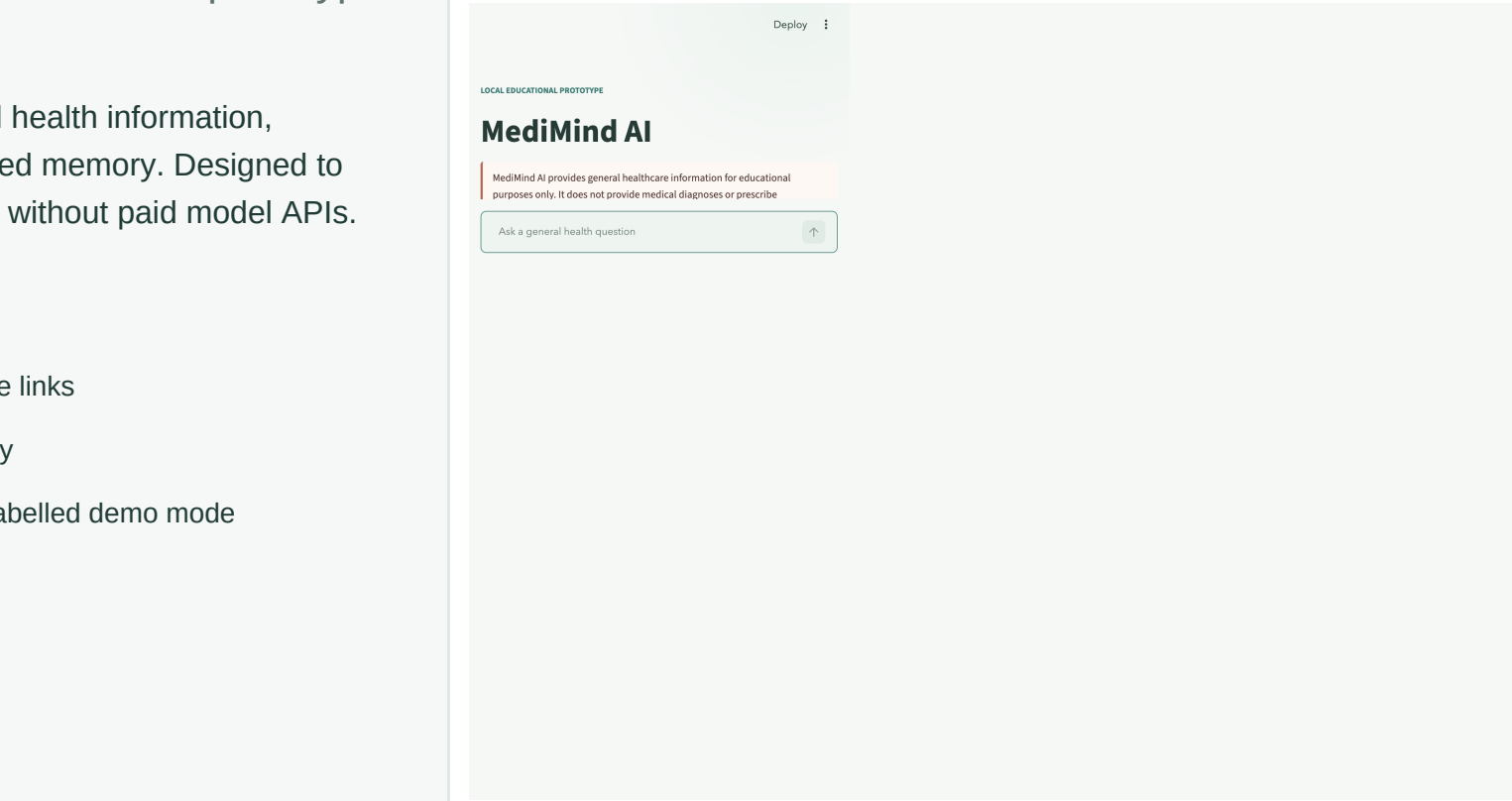
COLLEGE PROJECT OVERVIEW · 2026

MediMind AI

A local, agentic medical-information prototype

A proof of concept for educational health information, source retrieval, and patient-scoped memory. Designed to demonstrate local AI components without paid model APIs.

- Three sequential LangGraph agents
- Local ChromaDB retrieval with source links
- SQLite profile and consultation history
- Streamlit interface with an explicitly labelled demo mode



Captured from the running local app · response explicitly marked as demo text

What MediMind AI is

A compact college showcase of agentic information retrieval with local patient memory.

MediMind AI is a local-first educational assistant prototype. A user asks a general health-information question; the application can retrieve relevant excerpts from a small bundled reference set and, when Ollama is running, ask a local language model to form a cautious plain-language response.

CORE CAPABILITIES

- Chat UI with patient selection, agent status, and source references.
- Three sequential agents with explicit shared workflow state.
- Local semantic retrieval using sentence embeddings and ChromaDB.
- SQLite patient profiles and saved consultations, isolated by patient ID.
- Fixed-response demo mode when a live model is unavailable.

3
workflow agents

10
bundled references

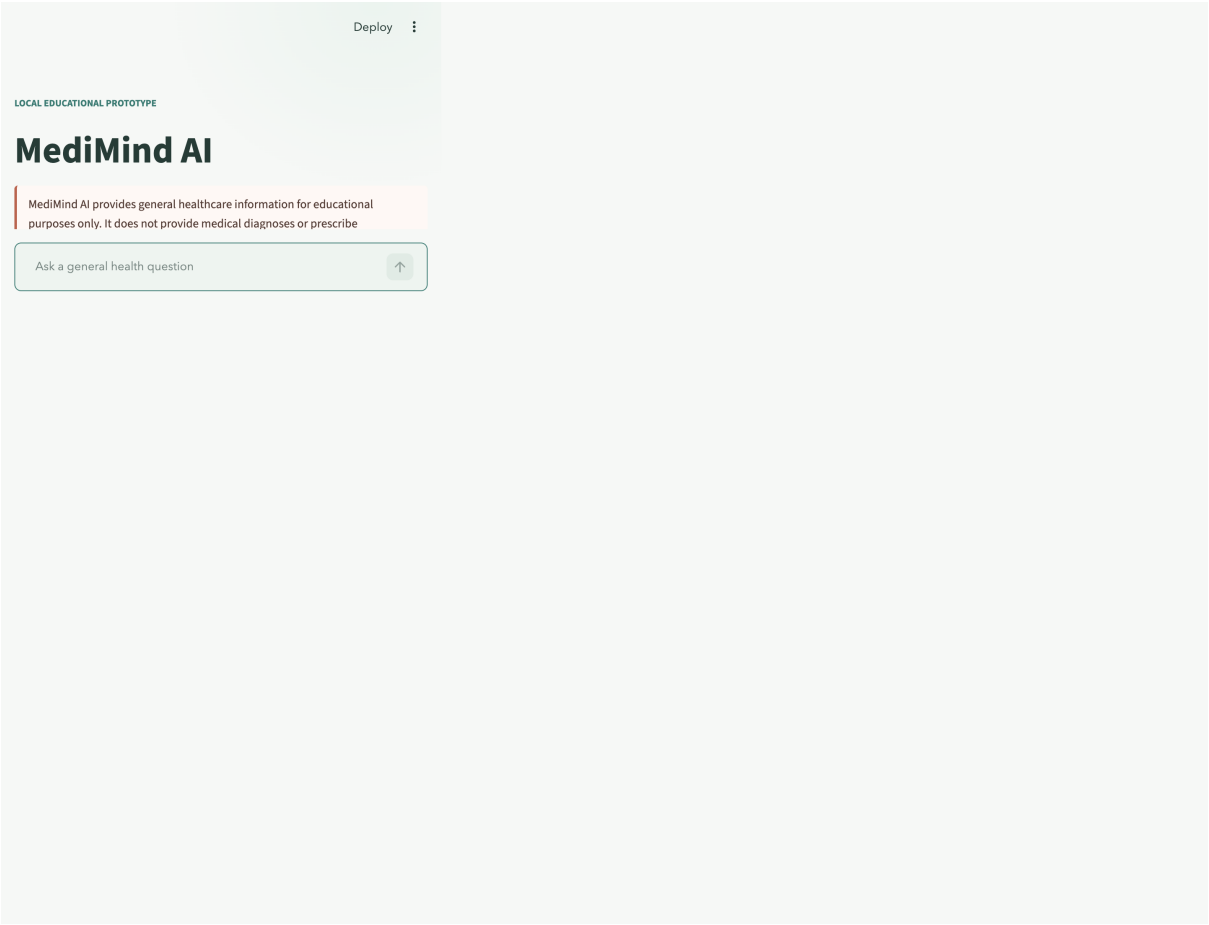
3
UI pages

0
paid AI APIs

Intended for a classroom demonstration and code exploration. It is not a clinical product.

Medical assistant

A chat-oriented interface keeps the question, result, execution state, and sources together.



Assistant page captured in labelled demo mode

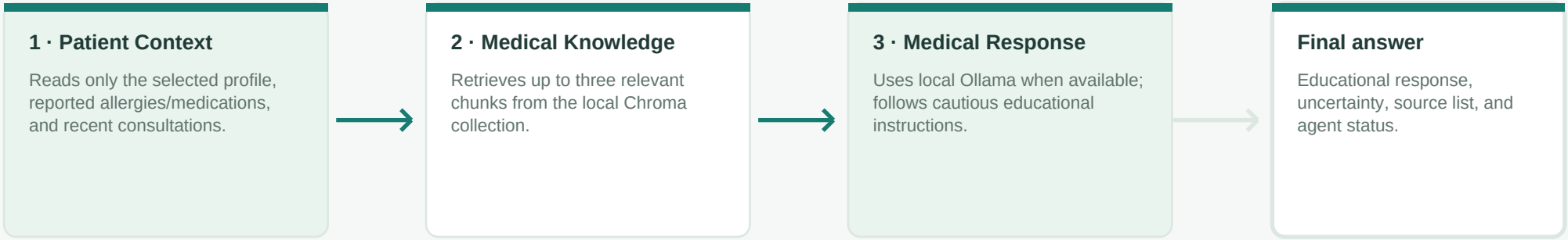
WHAT THE SCREEN DEMONSTRATES

- The response is clearly marked as fixed demo text, not live model output.
- Agent execution status is expandable for a concise, explainable trace.
- Retrieved references are displayed with titles and source names.
- Patient selection and history-saving confirmation are optional controls.

When demo mode is off and Ollama is unavailable, the app shows a local-service error and does not switch to a cloud API.

Three-agent workflow

A small sequential graph keeps responsibilities visible and easy to explain.



BEFORE THE GRAPH

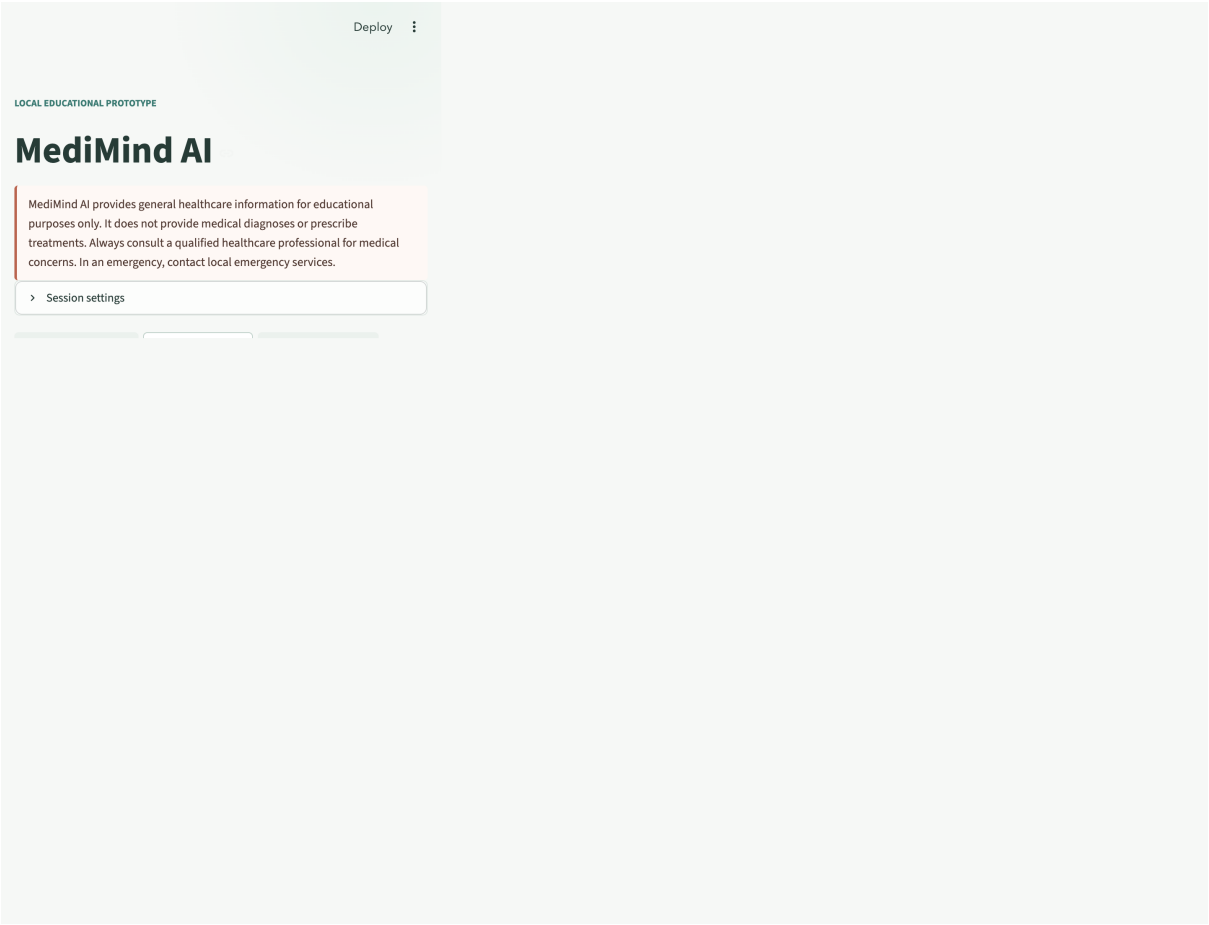
A basic emergency-keyword check runs first. Obvious emergency indicators bypass routine processing and return an urgent-care message.

SHARED STATE

The graph passes the question, selected patient ID, patient context, retrieved documents, response mode, answer, and per-agent status.

Patient profile and memory

Memory is deliberately simple: profile fields plus a chronological consultation table.



Patient profile form · captured with no patient information

STORED LOCALLY IN SQLITE

- Patient name and optional age.
- Medical history, reported allergies, and reported medications.
- Consultation question, assistant response, and timestamp.
- Recent consultations are included as context only for that selected patient.

The user confirms saving by enabling the history checkbox. Profile details are stored only after the profile form is submitted.

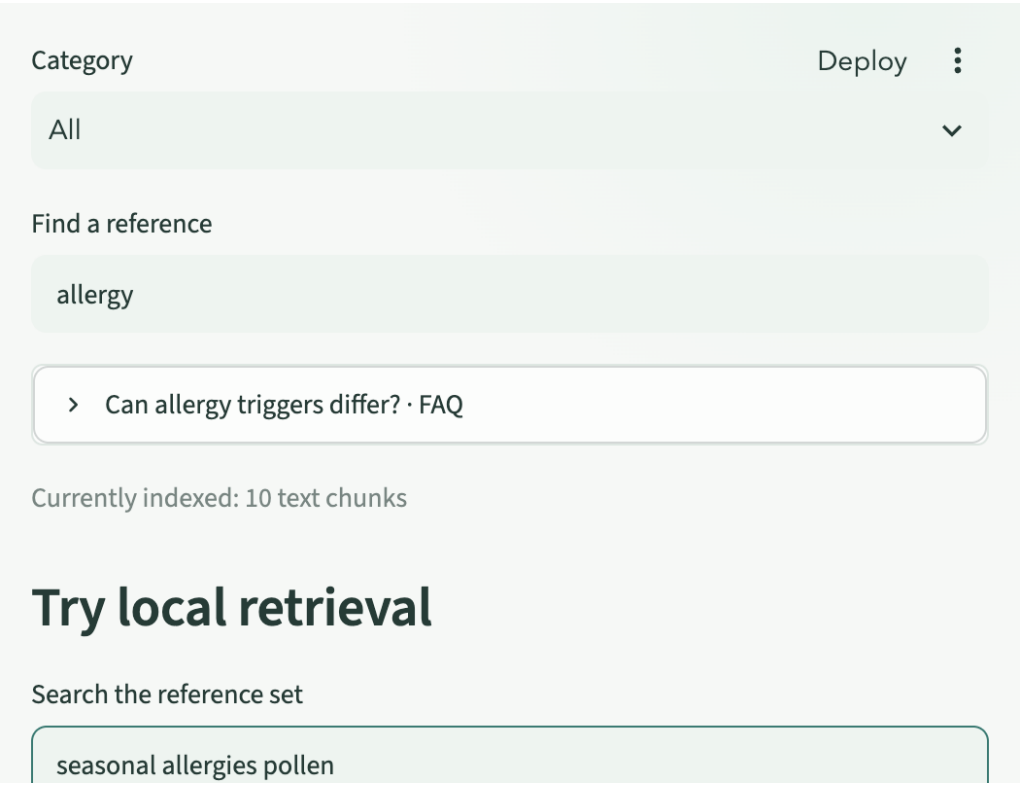
Knowledge base and sources

Bundled JSON references keep the demonstration independent of external medical datasets.

INCLUDED IN THIS BUILD

- 10 short educational documents across conditions, symptoms, FAQs, and medication safety.
- Source metadata includes document title, category, source name, and URL.
- Searchable library plus a retrieval demonstration in the UI.
- Verified query: seasonal allergies / pollen returned three matches, led by MedlinePlus allergic rhinitis.

The bundled set is intentionally small and is not a complete or clinically reviewed medical reference library.



Knowledge Base page · local query with top-three source results

How local retrieval works

The first embedding-model download is the only expected model-cache setup step for retrieval.

JSON
Load bundled records

CHUNK
Split searchable text

EMBED
MiniLM vectors locally

INDEX
Persist in ChromaDB

TOP 3
Return text + sources

10
indexed text chunks

MiniLM
all-MiniLM-L6-v2 · local embeddings

ChromaDB
persistent project-local collection

Observed in the local smoke test: model downloaded to the project cache, 10 chunks indexed, and top-three references returned with source metadata.

Data boundaries

Application packages, model caches, vector store, and user records are rooted in this project folder.

PROJECT-LOCAL STORAGE

- Python packages are installed in the workspace .venv.
- SQLite database: data/medimind.sqlite3.
- Chroma collection: data/chroma/.
- Embedding-model cache: data/models/.
- Streamlit home, cache, and temporary files are redirected to project-local folders at launch.

NETWORK BEHAVIOR

- Ollama requests are restricted to localhost.
- No paid LLM or embedding API is configured.
- The first embedding-model setup needs internet; cached retrieval can work offline afterward.
- Ollama itself is optional for the labelled demo mode and is not installed in the verified environment.

Local storage reduces external data transfer, but this prototype is not a hardened system for real patient records. Use synthetic data for demonstrations.

Guardrails and limitations

The system is designed to inform and demonstrate, not to diagnose or direct treatment.

IMPLEMENTED SAFEGUARDS

- Emergency keyword screen runs before the graph and advises contacting local emergency services.
- Prompt instructions prohibit diagnoses, prescriptions, dosage recommendations, and medication changes.
- A response filter withholds some obvious diagnostic or dosage instructions from generated text.
- Missing patient facts are marked as missing instead of being inferred.
- The required educational disclaimer is visible in the interface.
- Demo responses are fixed text and clearly identified as non-AI output.

IMPORTANT LIMITATION

Keyword checks and output filters are basic safeguards, not clinical validation. They can miss emergencies or unsafe wording. The local model may still produce inaccurate information.

Never rely on MediMind for medical decisions. In an emergency, contact local emergency services.

What runs locally

A lightweight local chat model is configured; retrieval uses a separate small embedding model.

Python
3.11.7 in .venv

Streamlit
UI · 1.66.1

LangGraph
workflow · 1.2.19

ChromaDB
vector store · 1.5.9

Sentence Transformers
embeddings · 5.7.0

SQLAlchemy + SQLite
patient memory · 2.1.6

CHAT MODEL STATUS

Default Ollama model: smollm2:360m. This was selected for low resource use. Ollama is not installed here, so live generation has not been verified. The app reports an unavailable error rather than falling back to an external service.

What was tested

A distinction between automated checks, manual smoke tests, and unverified live-model behavior.

9 / 9	3 / 3	10	3
automated tests passed	agents reported in workflow test	RAG chunks indexed	top references returned

VERIFIED

- SQLite profile creation, retrieval, isolation, and consultation persistence.
- JSON loading, chunking, and Chroma retrieval with mocked embeddings in tests.
- Emergency short-circuit, three-agent workflow, demo mode, and local-host restriction.
- Manual browser checks of assistant, profile, and knowledge-base pages.
- Real local MiniLM retrieval with source attribution.

NOT VERIFIED / NOT INSTALLED

- Live Ollama inference: Ollama runtime and model were absent.
- Clinical quality or completeness: this is a small educational corpus, not clinical validation.
- The exact emergency keyword list is not a replacement for triage.

Run locally and next steps

A short classroom walkthrough can demonstrate the architecture without model service access.

DEMO SEQUENCE

- Launch the app from the project .venv.
- Turn on labelled demo mode and submit a general question.
- Expand agent status and source references.
- Open Patient profile; create synthetic data and explicitly confirm history saving.
- Open Knowledge Base; filter a topic and run a local retrieval query.
- Try an emergency keyword to show the pre-graph safety response.

IF ENABLING LIVE GENERATION

- Install Ollama separately and keep its model directory under data/ollama/models.
- Pull the configured small model (smollm2:360m) or set OLLAMA_MODEL to another local model.
- Start Streamlit with project-local HOME, cache, and temp directories as documented in README.md.

Bottom line: MediMind demonstrates local orchestration, retrieval, persistence, and a cautious UI. It is a teaching prototype, not a medical service.